

Identifying Key Entities in Recipe Data

Objectives

The goal of this assignment is to train a Named Entity Recognition (NER) model using Conditional Random Fields (CRF) to extract key entities from recipe data. The model will classify words into predefined categories such as ingredients, quantities and units, enabling the creation of a structured database of recipes and ingredients that can be used to power advanced features in recipe management systems, dietary tracking apps, or e-commerce platforms.

Problem Statement

Recipes are typically written in unstructured natural language, making it difficult to automatically extract meaningful information such as ingredients, quantities, and measurement units. This lack of structure limits the ability to build intelligent applications that rely on precise recipe data.

The objective of this assignment is to develop a **Named Entity Recognition (NER) model** using **Conditional Random Fields (CRF)** to identify and classify key entities in recipe text. Each token in a recipe will be labeled into predefined categories such as **ingredient**, **quantity**, and **unit**.

By accurately extracting these entities, the model will transform raw recipe text into a structured format. This structured data can then be used to support advanced use cases including recipe management systems, dietary and nutrition tracking applications, grocery planning tools, and e-commerce platforms.

The key challenge addressed in this problem is handling the variability and ambiguity of natural language in recipes, where the same word may serve different roles depending on context. The CRF model leverages contextual and sequential information to improve entity recognition accuracy and robustness.

Data Description

Key	Description
input	Contains a raw ingredient list from a recipe.
pos	Represents the corresponding part-of-speech (POS) tags or NER labels, identifying quantities, ingredients, and units.

Data Preparation

The data preparation part of the notebook involves a few steps:

- **Importing Libraries:**
 - **Installation of sklearn-crfsuite:** sklearn-crfsuite is a Python wrapper for CRFsuite, a fast and efficient implementation of Conditional Random Fields (CRFs). It is designed to integrate seamlessly with scikit-learn for structured prediction tasks such as Named Entity Recognition (NER), Part-of-Speech (POS) tagging, and chunking.
 - Imports the warnings module and then sets a filter to 'ignore' all warnings that might be generated during the execution of the code. This is often done to prevent warning messages from cluttering the output, especially in development or demonstration environments
 - Imports a comprehensive set of Python libraries for a machine learning project, likely focused on Natural Language Processing (NLP) and Named Entity Recognition (NER) using Conditional Random Fields (CRF). It covers:
 - Data Handling: json for JSON data, pandas for DataFrame operations, re for regular expressions, numpy for numerical computations.
 - Visualization: matplotlib.pyplot and seaborn for creating plots and charts.
 - NLP & Machine Learning: spacy for advanced NLP tasks, sklearn_crfsuite for CRF model implementation, and sklearn.model_selection, sklearn.metrics, sklearn.utils.class_weight for model training, evaluation, and handling class imbalances.
 - Utilities: joblib for saving/loading models, random for randomness, fractions for handling fractions, collections.Counter for frequency counting, and IPython.display for enhanced output in notebooks.
- **Data Ingestion and Preparation:**
 - **Read Recipe Data from Dataframe and prepare the data for analysis**
 - **Define a load_json_dataframe function**
 - Defines a function load_json_dataframe that takes a file path as input
 - Opens the specified JSON file in read mode safely using a with statement
 - Loads and parses the JSON content into a Python object
 - Converts the loaded JSON data into a pandas DataFrame
 - Returns the DataFrame for further data analysis or processing
 - **Execute the load_json_dataframe function**
 - Defines the file path pointing to the JSON file stored in Google Drive
 - Calls the load_json_dataframe function with this file path
 - Reads the JSON file and converts its contents into a pandas DataFrame
 - Stores the resulting DataFrame in the variable df for further analysis

- **Describe the dataframe**

```
RangeIndex: 285 entries, 0 to 284
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0   input    285 non-null    object
1   pos      285 non-null    object
dtypes: object(2)
```

- The DataFrame contains 285 total rows
- Both columns (input and pos) have 285 non-null values
- This confirms that no missing or null values are present in either column
- The dataset is complete and ready for further processing or analysis

- **Recipe Data Manipulation**

- **Create input_tokens and pos_tokens columns by splitting the input and pos from the dataframe**

- The input column is tokenized by splitting each sentence into a list of individual words and stored in input_tokens
- The pos column is tokenized by splitting each sequence of POS tags into a list and stored in pos_tokens
- Tokenization converts long text strings into structured lists, which are easier to process
- This step prepares the data for NLP tasks such as tagging, parsing, or entity recognition

- **Provide the length for input_tokens and pos_tokens and validate their length**

- Creates an input_length column to store the number of words in each input_tokens list
- Creates a pos_length column to store the number of tags in each pos_tokens list
- These columns help validate alignment between input tokens and POS tags
- Rows with unequal lengths can be identified to detect data inconsistencies or errors before further NLP processing

Based on the previous validation step, the rows that had unequal input and POS token lengths are at the following indices: 17, 27, 79, 164, 207.

- **Define a unique_labels function and validate the labels in pos_tokens**
 - Defines a unique_labels function to identify all distinct POS labels in the dataset
 - Iterates through the pos_tokens column and collects all POS tags
 - Removes duplicates by converting the list into a set and sorts the labels
 - Returns and prints the unique POS labels present in the recipes
 - Confirms that the dataset contains three labels: ingredient, quantity, and unit
- **Drop the rows that have invalid data provided and validate**
 - Identifies and drops rows with invalid data using predefined indices
 - Resets the DataFrame index to maintain a clean, continuous index
 - Recalculates input_length and pos_length after removing invalid rows
 - Updates these columns to reflect the current token counts
 - Compares input_length and pos_length to validate token–tag alignment
 - Confirms whether any rows have unequal lengths or verifies that all rows are consistent

Train Validation Split (70 train - 30 val)

- **Perform train and validation split ratio**
 - **Split the dataset into train_df and val_df into 70:30 ratio**
 - Splits the cleaned dataset into training and validation sets
 - Uses a 70:30 ratio, where 70% of data is for training and 30% for validation
 - Sets a fixed random_state to ensure reproducibility of the split
 - Helps evaluate model performance on unseen data during validation
 - **Extract the dataset into train_df and val_df into X_train, X_val, y_train and y_val and display their length**
 - Extracts tokenized input sequences (input_tokens) from train_df and val_df into X_train and X_val
 - Extracts corresponding POS tag sequences (pos_tokens) into y_train and y_val
 - Converts DataFrame columns into Python lists suitable for model training
 - Prepares separate datasets for training and validation
 - Prints and verifies the number of samples in each dataset

- Ensures that feature and label lengths are consistent and aligned before model training
- **Display the number of unique labels present in y_train**
 - Flattens the nested POS tag sequences in y_train into a single list
 - Uses a set to identify unique labels present in the training data
 - Counts the total number of unique POS labels
 - Displays both the number of unique labels and their sorted names
 - Confirms label diversity in the training dataset before model training

Unique labels in y_train: 3

Labels: ['ingredient', 'quantity', 'unit']

Exploratory Recipe Data Analysis on Training Dataset

- **Flatten the lists for input_tokens & pos_tokens**
 - Defines a function to flatten a nested list into a single list
 - Takes a list of lists as input
 - Iterates through each sublist and extracts individual elements
 - Combines all elements into one flat structure
 - Returns the flattened list for easier processing
 - Labels the dataset as Training to indicate the context of use
- **Extract and validate the tokens after using the flattening technique**
 - **Flattening:** It converts nested lists of tokens (organized by sentence or row) into a single, continuous list. This reshapes the data into a format suitable for global analysis and model training.
 - **Parallel Extraction:** The function simultaneously flattens both the raw words (input_tokens) and their corresponding labels (pos_tokens) to ensure they stay in the exact same order.
 - **Alignment Validation:** It performs a critical check to ensure the total number of words matches the total number of tags. This prevents "offset errors" where a word might be paired with the wrong label.
 - **Descriptive Logging:** By passing a dataset_name, the function provides clear, labeled feedback in the console, making it easy to track whether the Training or Validation data was processed successfully.

- **Sanity Sampling:** It displays the first 10 records as a quick visual confirmation that the mapping is correct (e.g., verifying that "250" is correctly identified as a "quantity").
- **Output for Downstream Tasks:** It returns the two flattened lists, serving as the final preparation step before the data is fed into a machine learning pipeline or vectorizer.

Extracted the tokens and pos tags -

First 10 input tokens (Training): ['250', 'grams', 'Okra', 'Oil', '1', 'Onion', 'finely', 'chopped', 'Tomato', 'Grated']

First 10 POS tokens (Training): ['quantity', 'unit', 'ingredient', 'ingredient', 'quantity', 'ingredient', 'ingredient', 'ingredient', 'ingredient', 'ingredient']

- **Categorise tokens into labels (unit, ingredient, quantity)**
 - Iterates through the flattened lists to separate tokens based on their specific POS tags.
 - Filters tokens into three distinct categories: Ingredients, Units, and Quantities.
 - Transforms raw labeled data into structured, theme-specific lists for easier analysis.
 - **Parallel Processing:** Uses the 1:1 relationship between tokens and tags to ensure each word is placed in the correct bucket.
 - Returns three individual lists, allowing for independent processing of quantities, measurements, and items.
- **Top 10 Most Frequent Items**
 - **Frequency Mapping:** Uses the Counter object to automatically tally occurrences of every unique item in the provided list.
 - **Ranking:** Employs the `most_common(n)` method to identify and sort the items with the highest counts in descending order.
 - **Contextual Reporting:** Utilizes parameters like `label_type` and `dataset_name` to generate clear, labeled headers for the output.
 - **Structured Formatting:** Iterates through the results to display the top 10 items in a clean, readable list format.
 - **Reusable Logic:** Designed as a general-purpose function that can be applied to any category (ingredients, units, or quantities) across different data splits.
 - **Data Return:** Returns the results as a list of tuples, enabling the data to be used later for visualization or further statistical analysis.

Top 10 Most Frequent Ingredients (Training Data):

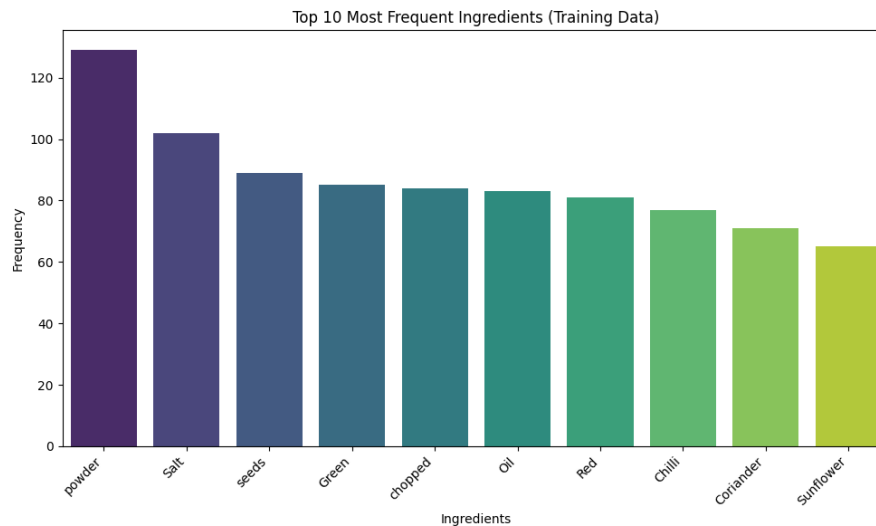
- powder: 129
- Salt: 102
- seeds: 89
- Green: 85
- chopped: 84
- Oil: 83
- Red: 81
- Chilli: 77
- Coriander: 71
- Sunflower: 65

Top 10 Most Frequent Units (Training Data):

- teaspoon: 162
- cup: 136
- tablespoon: 99
- grams: 63
- tablespoons: 61
- inch: 52
- cups: 50
- sprig: 41
- cloves: 39
- teaspoons: 39

- **Plot Top 10 most frequent items**

- Top 10 Most Frequent Ingredients



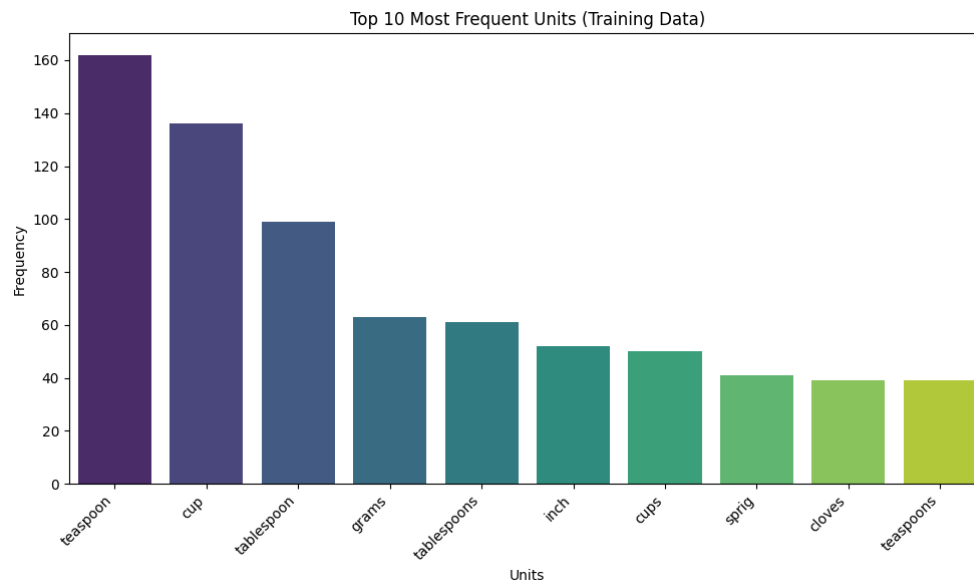
Based on the top 10 most frequent ingredients from both the training and validation datasets, here are some key insights:

- **Consistency Across Datasets:** The top ingredients are remarkably consistent between the training and validation sets. Words like 'powder', 'Salt', 'seeds', 'Green', 'chopped', 'Oil', 'Red', 'Chilli', and 'Coriander' appear in both lists, often in similar relative ranks. This indicates that the dataset is representative and the split (70:30) has maintained the underlying distribution of ingredient types.
- **Common Ingredient Modifiers/Types:** Many of the most frequent 'ingredients' are actually modifiers or general categories rather than specific ingredients. For example:
 - 'powder': This is a common form for many spices (e.g., turmeric powder, chili powder).
 - 'seeds': Refers to various seeds used in cooking (e.g., cumin seeds, mustard seeds).
 - 'Green' / 'Red': Often used to describe chilies or other vegetables.
 - 'chopped': A preparation method for ingredients.
 - 'Oil' / 'Salt' / 'Chilli' / 'Coriander': These are fundamental and widely used components in a vast majority of recipes.
- **Dominance of Spices and Basic Seasonings:** The prevalence of 'powder', 'Salt', 'seeds', 'Chilli', and 'Coriander' suggests that a significant portion of the recipe data contains instructions for dishes that heavily rely on spices and basic seasonings.
- **Preparation Terms as Ingredients:** Terms like 'chopped' are categorized as 'ingredient', which might indicate a nuanced aspect of the POS tagging. While 'chopped' describes an

ingredient, in this context, it appears as an ingredient itself within the tokenization, reflecting how recipes often list items.

Overall, the ingredient plots confirm that the recipe data contains common cooking staples and preparation forms, and that the model is being trained and validated on a consistent representation of these elements.

- Top 10 Most Frequent Units



Based on the top 10 most frequent units from both the training and validation datasets, here are some key insights:

- **Dominance of Volume Measurements:** 'teaspoon', 'cup', and 'tablespoon' (along with their plural forms) are consistently the most frequent units. This indicates that a large proportion of recipes use these standard volume measurements, which is expected for many cooking instructions.
- **Consistency Between Datasets:** Similar to ingredients, the top units show remarkable consistency between the training and validation sets. The same units appear in both lists, and their relative ranks are largely maintained, suggesting a stable distribution of unit types across the dataset.
- **Weight Measurements Present:** 'grams' also appears in the top units, indicating that some recipes specify ingredients by weight, which is common in baking or more precise culinary contexts.
- **Units for Specific Items/Preparation:** Units like 'inch', 'sprig', and 'cloves' are present, which are used for specific types of ingredients or presentations (e.g., 'an inch of ginger', 'a sprig of rosemary', 'cloves of garlic'). Their inclusion suggests that the dataset captures a good variety of ingredient descriptions.

- Pluralization: Both singular and plural forms of units ('teaspoon'/'teaspoons', 'cup'/'cups', 'tablespoon'/'tablespoons') are recognized and present in the top lists, highlighting the model's ability to handle these variations.

Overall, the unit plots confirm that the recipe data contains a diverse yet common set of measurement units, primarily focusing on volume, with consistent representation across the training and validation splits. This provides a solid foundation for the NER model to learn and extract these entities effectively.

Exploratory Recipe Data Analysis on Validation Dataset

Flattened token lengths are equal for Validation data: 2876.

First 10 input tokens (Validation): ['1', 'cup', 'Ada', '2', 'liter', 'Milk', '3/4', 'Sugar', 'tablespoon', 'Ghee']

First 10 POS tokens (Validation): ['quantity', 'unit', 'ingredient', 'quantity', 'unit', 'ingredient', 'quantity', 'ingredient', 'unit', 'ingredient']

Top 10 Most Frequent Ingredients (Validation Data):

- powder: 54
- Salt: 47
- Oil: 39
- Red: 39
- seeds: 36
- Chillli: 36
- chopped: 31
- Green: 30
- Leaves: 29
- Coriander: 27

Top 10 Most Frequent Units (Validation Data):

- teaspoon: 59
- cup: 57
- tablespoon: 32
- tablespoons: 32
- cups: 24
- sprig: 21
- inch: 20
- grams: 19
- teaspoons: 18
- cloves: 16

Here are the insights from the provided validation data EDA output:

- **Data Consistency:** The message "Flattened token lengths are equal for Validation data: 2876" confirms that after processing, the number of input tokens and their corresponding POS tokens perfectly match for the validation set. This indicates clean and consistent data, which is crucial for model training.
- **Sample Token Structure:** The first 10 input tokens and their POS tags (['1', 'cup', 'Ada', '2', 'liter', 'Milk', '3/4', 'Sugar', 'tablespoon', 'Ghee'] and ['quantity', 'unit', 'ingredient', 'quantity', 'unit', 'ingredient', 'quantity', 'ingredient', 'unit', 'ingredient']) illustrate how quantities, units, and ingredients are tokenized and labeled, showcasing the model's target output structure.

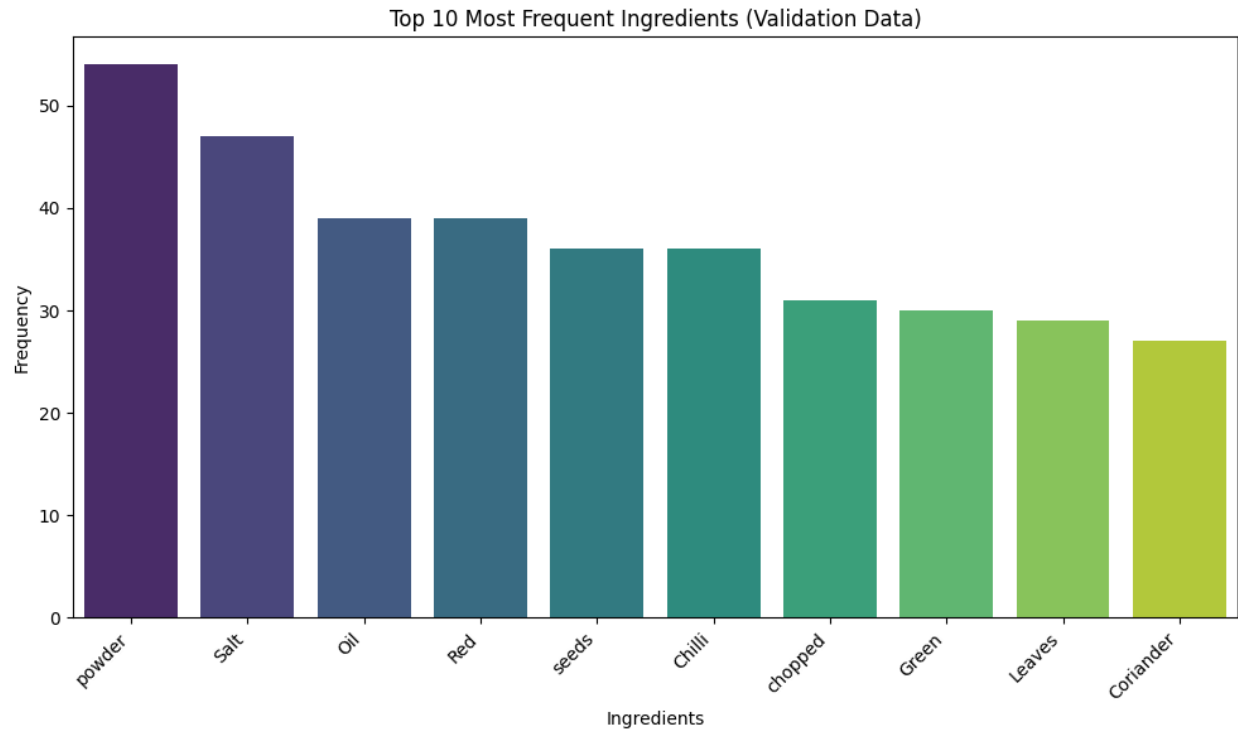
Top 10 Most Frequent Ingredients (Validation Data):

- The list includes common cooking components and descriptors like 'powder', 'Salt', 'Oil', 'seeds', 'Chilli', and 'Coriander'. Terms like 'Red', 'chopped', and 'Green' also appear frequently, indicating that these descriptive words are often tagged as ingredients within this dataset. This distribution is very similar to what was observed in the training data, reinforcing the consistency and representativeness of the dataset split.

Top 10 Most Frequent Units (Validation Data):

- Volume measurements such as 'teaspoon', 'cup', and 'tablespoon' (including their plural forms) are the most dominant units. Other units like 'sprig', 'inch', 'grams', and 'cloves' are also present, demonstrating a variety of measurement types beyond just volume. Again, the patterns here closely mirror those in the training data, indicating that the validation set provides a consistent test of the model's ability to recognize common units.

- Top 10 Most Frequent Ingredients(Validation Data)

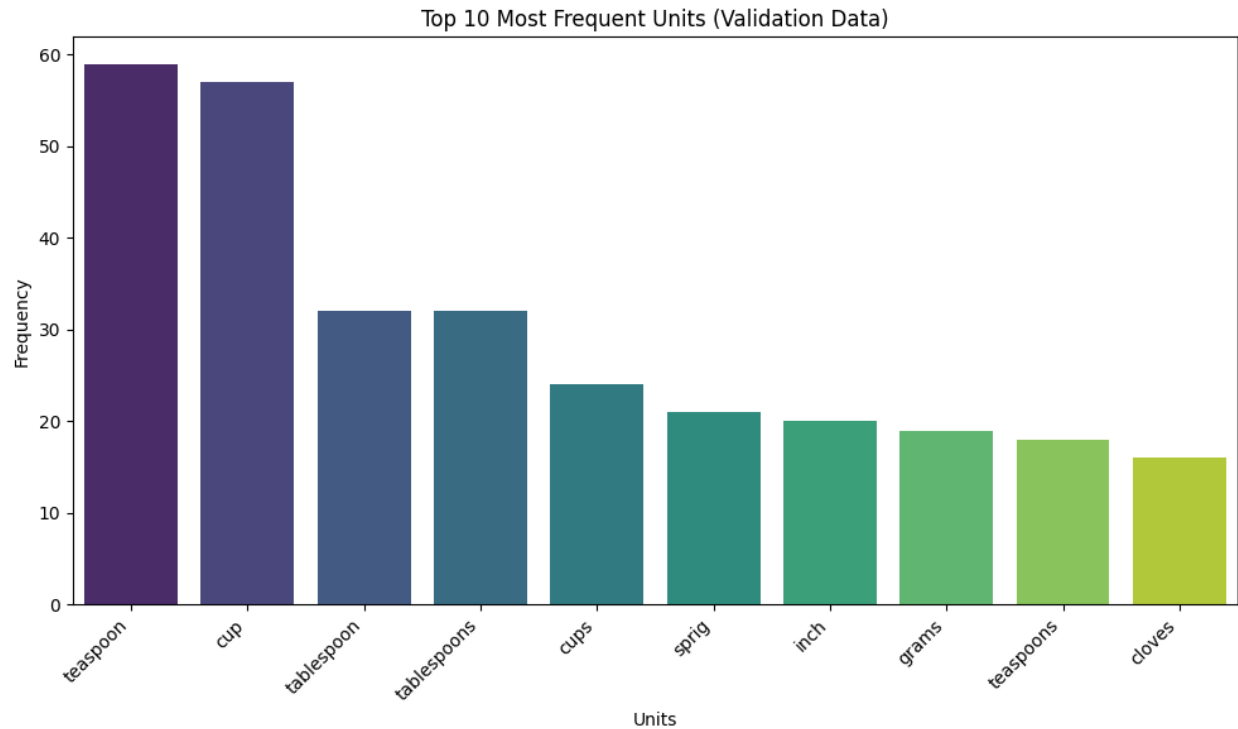


Insights from the validation dataset regarding ingredients:

Ingredient Insights:

- **Consistency:** The top ingredients in the validation set (e.g., 'powder', 'Salt', 'Oil', 'seeds', 'Chilli', 'Coriander') closely mirror those in the training set, confirming a representative data split.
- **Common Categories:** Frequently seen 'ingredients' often include basic seasonings, spices, and descriptive terms like 'chopped', which are tagged as ingredients within this dataset.

- Top 10 Most Frequent Units(Validation Data)



Unit Insights:

- Dominance of Volume: 'teaspoon', 'cup', and 'tablespoon' (including plurals) are the most prevalent units, indicating a strong focus on volume measurements in recipes.
- Consistency: Similar to ingredients, the unit distribution is consistent with the training data.
- Diversity: The presence of units like 'grams', 'sprig', and 'inch' shows that the dataset captures a variety of measurement types beyond just volume.

Feature Extraction For CRF Model

- **Define a feature functions to take each token from recipe**
 - unit_keywords: This is a Python set containing a comprehensive list of words that commonly represent units of measurement in recipes. It includes:
 - Volume units: 'cup', 'cups', 'teaspoon', 'teaspoons', 'tablespoon', 'tablespoons', 'ml', 'liter', 'liters', 'dl', 'tbsps', 'tsp'.
 - Weight units: 'g', 'kg', 'lb', 'oz', 'ounce', 'ounces', 'gram', 'grams', 'gms'.
 - Count/Descriptive units: 'pinch', 'can', 'cans', 'bottle', 'bottles', 'clove', 'cloves', 'slice', 'slices', 'sprig', 'sprigs', 'dash', 'dashes', 'packet', 'packets', 'piece', 'pieces', 'drop', 'drops', 'stalk', 'stalks', 'head', 'heads', 'sheets'.

- Length units: 'inch', 'inches', 'cms'.
- The use of a set allows for fast lookup when checking if a word is a unit.
- `quantity_keywords`: This is another Python set that stores descriptive words and modifiers often associated with quantities or the state of an ingredient, but are not necessarily numerical units themselves. It includes terms like:
 - Amount descriptors: 'half', 'quarter', 'double', 'single', 'few', 'some', 'lot', 'little', 'small', 'medium', 'large', 'a', 'an'.
 - Preparation/State descriptors: 'fresh', 'dried', 'frozen', 'chopped', 'minced', 'sliced', 'diced', 'grated', 'crushed', 'finely', 'coarsely', 'melted', 'warm', 'cold', 'hot', 'boiling', 'softened', 'beaten', 'whipped', 'squeezed', 'packed', 'firmly', 'loosely', 'drained', 'rinsed', 'peeled', 'seeded', 'deveined', 'shelled', 'toasted', 'roasted', 'ground', 'whole', 'cut', 'raw', 'cooked', 'ripe', 'unripe', 'liquid', 'solid', 'powdered', 'granulated', 'cubed', 'shredded', 'julienned', 'halved', 'quartered'.
 - Instructional/Advisory terms: 'each', 'about', 'approx', 'approximately', 'heaped', 'level', 'rounded', 'to taste', 'as needed', 'optional', 'extra', 'light', 'dark', 'sweet', 'sour', 'bitter', 'spicy', 'mild', 'hot', 'room temperature'.
- These keywords help identify non-numeric quantity information.
- `quantity_pattern`: This is a compiled regular expression (`re.compile`) designed to match various numerical formats that represent quantities. It's updated to handle a wide range of patterns:
 - `^(\d+\s+)?`: Optionally matches a whole number followed by a space (e.g., '1 ', '2 '). This handles cases like '1 1/2' or '2 3/4'.
 - `(\d*.\d+|\d+/?\d*|\d*\s*\d+)`: This is the core part, matching one of three numeric formats:
 - `\d*.\d+`: Matches decimal numbers (e.g., '0.5', '.5', '2.5').
 - `\d+/?\d*`: Matches whole numbers (e.g., '5', '12') or fractions (e.g., '1/2', '3/4').
 - `\d*\s*\d+`: Matches number ranges (e.g., '2-3', '10 - 12').
 - `([/.\d]*)*$`: Optionally allows for additional fractional or decimal parts at the end, covering complex fractional or range structures that might extend (though the previous part is usually sufficient).
 - The `^` and `$` ensure the pattern matches the entire string.
 - This regex is crucial for identifying quantities that are expressed numerically, including mixed numbers and ranges.

- **Define feature functions for CR**

- While building features dictionary, include
 - Core Features - The core features of a token should capture its lexical and grammatical properties. Include attributes like the raw token, its lemma, part-of-speech tag, dependency relation, and shape, as well as indicators for whether it's a stop word, digit, or punctuation. The details of the features are given below:
 - bias - Constant feature with a fixed value of 1.0 to aid model learning.
 - token - The lowercase form of the current token.
 - lemma - The lowercase lemma (base form) of the token.
 - pos_tag - Part-of-speech (POS) tag of the token.
 - tag - Detailed POS tag of the token.
 - dep - Dependency relation of the token in the sentence.
 - shape - Shape of the token (e.g., "Xxx" for "Milk").
 - is_stop - Boolean indicating if the token is a stopword.
 - is_digit - Boolean indicating if the token consists of only digits.
 - has_digit - Boolean indicating if the token contains at least one digit.
 - has_alpha - Boolean indicating if the token contains at least one alphabetic character.
 - hyphenated - Boolean indicating if the token contains a hyphen (-).
 - slash_present - Boolean indicating if the token contains a slash (/).
 - is_title - Boolean indicating if the token starts with an uppercase letter.
 - is_upper - Boolean indicating if the token is fully uppercase.
 - is_punct - Boolean indicating if the token is a punctuation mark.
 - Improved Quantity and Unit Detection - Use key-value pairs to mark the presence of quantities and units in the features dictionary. Utilise the unit_keywords, quantity_keywords, and quantity_pattern to identify and flag these elements. The details of the features are given below:
 - is_quantity - Boolean indicating if the token matches a quantity pattern or keyword.
 - is_unit - Boolean indicating if the token is a known measurement unit.
 - is_numeric - Boolean indicating if the token matches a numeric pattern.
 - is_fraction - Boolean indicating if the token represents a fraction (e.g., 1/2).
 - is_decimal - Boolean indicating if the token represents a decimal number (e.g., 3.14).
 - preceding_word - The previous token in the sentence, if available.

- following_word - The next token in the sentence, if available.
 - Contextual Features - Incorporate contextual information by adding features for the preceding and following tokens. Include indicators like BOS and EOS to mark the beginning and end of the sequence, and utilise unit_keywords, quantity_keywords, and quantity_pattern to identify the types of neighboring tokens. The features are given below:
 - prev_token - The lowercase form of the previous token.
 - prev_is_quantity - Boolean indicating if the previous token is a quantity.
 - prev_is_digit - Boolean indicating if the previous token is a digit.
 - BOS - Boolean indicating if the token is at the beginning of the sentence.
 - next_token - The lowercase form of the next token.
 - next_is_unit - Boolean indicating if the next token is a unit.
 - next_is_ingredient - Boolean indicating if the next token is not a unit or quantity.
 - EOS - Boolean indicating if the token is at the end of the sentence.
 - The word2features function is designed to extract a rich set of features for a given word within a sentence, crucial for Named Entity Recognition (NER). It creates a dictionary of features for each token, encompassing three main categories:
 - Core Lexical and Grammatical Features: Extracts properties of the current token using spaCy, such as its lowercase form, lemma, POS tags, dependency relation, shape, and boolean indicators for stop words, digits, punctuation, title case, and uppercase.
 - Improved Quantity and Unit Detection: Includes boolean flags to identify if the current token is a quantity (based on quantity_keywords or quantity_pattern), a unit (from unit_keywords), or other numeric/fractional/decimal forms.
 - Contextual Features: Adds information about the preceding and succeeding tokens, including their lowercase forms, and whether they are quantities, units, or digits. It also marks the beginning (BOS) and end (EOS) of the sentence.
- **Preparation of Recipe level features**
 - Define sent2features function and inputs sent as a parameter and correctly generate feature functions for each token present in the sentence
 - sent2features by working on each token in the sentence and correctly generate dictionaries for features

- Convert `X_train`, `X_val`, `y_train` and `y_val` into train and validation feature sets and labels
- Convert labels of `y_train` and `y_val` into list
- Flattens the nested `y_train` structure into a single list `y_train_flat` containing all POS labels
- Ensures all training labels are in a one-dimensional format for counting and analysis
- Counts the frequency of each label using `Counter`, stored in `label_counts`
- Computes the total number of label samples as `total_samples`
- Calculates class weights using the inverse frequency method, assigning higher weights to rarer labels
- Stores these weights in a dictionary called `weight_dict`
- Applies an additional penalty to the ingredient label by reducing its weight
- Encourages the model to focus more on less frequent labels (quantity, unit)
- Helps address class imbalance and improves model training effectiveness
- Extract features along with class weights
 - The function augments token-level features with class-specific weights for CRF training
 - Helps address class imbalance by emphasizing less frequent labels
 - Takes three inputs: tokenized sentences (`X`), corresponding labels (`y`), and a label-to-weight mapping (`weight_dict`)
 - Iterates through each sentence and its associated label sequence
 - Generates base features for each token using an existing feature extraction method
 - Adds a new feature called `label_weight` to each token's feature set
 - Assigns the weight based on the token's true label using `weight_dict`
 - Uses a default weight of 1.0 if a label is not found in the dictionary
 - Returns a structured list of feature dictionaries per sentence
 - Enables the CRF model to prioritize important or rare entity labels during training
- Execute `extract_features_with_class_weights` on training and validation datasets
 - Applies class-weighted feature extraction to the **training dataset** using predefined label weights
 - Enhances each token's feature set with a `label_weight` based on its true label
 - Stores the weighted features for training in `X_train_weighted_features`
 - Repeats the same weighted feature extraction process for the **validation dataset**
 - Ensures consistency by using the same `weight_dict` for both training and validation
 - Prepares both datasets in a format suitable for **CRF model training and evaluation**

- Helps the CRF model handle **class imbalance** by emphasizing more important or rare labels

Model Building and Training

- **Initialise the CRF model and train it**

CRF Model Hyperparameters Explanation

Parameter	Description
algorithm='lbfgs'	Optimisation algorithm used for training. <code>lbfgs</code> (Limited-memory Broyden–Fletcher–Goldfarb–Shanno) is a quasi-Newton optimisation method.
c1=0.5	L1 regularisation term to control sparsity in feature weights. Helps in feature selection.
c2=1.0	L2 regularisation term to prevent overfitting by penalising large weights.
max_iterations=100	Maximum number of iterations for model training. Higher values allow more convergence but increase computation time.
all_possible_transitions=True	Ensures that all possible state transitions are considered in training, making the model more robust

- **Evaluation of Training Dataset using CRF model**

This training classification report shows excellent performance on the training dataset. Here are the key insights:

- **Perfect Precision (1.00 for all classes):** This means that when the model predicted a token to be an 'ingredient', 'quantity', or 'unit', it was always correct. There were no false positives for any of these categories on the training data.
- **Perfect Recall (1.00 for all classes):** This indicates that the model successfully identified every single 'ingredient', 'quantity', and 'unit' token present in the training data. There were no false negatives, meaning no actual entities were missed.
- **Perfect F1-score (1.00 for all classes):** The F1-score is the harmonic mean of precision and recall. A score of 1.00 signifies that the model achieved a perfect balance of precision and recall for all categories, identifying all entities correctly without any mistakes.
- **Support:** The 'support' column indicates the total number of actual occurrences of each label in the training dataset: 5323 'ingredient' tokens, 980 'quantity' tokens, and 811 'unit' tokens.

- Overall Accuracy (1.00): The model correctly classified every single token in the training set.
- Macro and Weighted Averages: Both macro and weighted averages are 1.00, which is expected when individual class scores are all 1.00.

Overall Insight: The model has achieved a perfect fit on the training data. While this shows that the model is highly capable of learning the patterns in the data, such perfect scores on the training set often raise a red flag for overfitting. It's crucial to evaluate the model's performance on a separate, unseen validation dataset to determine if it can generalize well to new data or if it has simply memorized the training examples.

- **Training Confusion Matrix**

	ingredient	quantity	unit
ingredient	5323	0	0
quantity	0	977	3
unit	0	0	811

This training confusion matrix provides a detailed view of the model's performance on the training dataset. Here are the key insights:

- Perfect Classification for 'ingredient': All 5323 tokens that were truly 'ingredient' were correctly classified as 'ingredient'. There were no misclassifications for this category.
- Excellent Classification for 'unit': All 811 tokens that were truly 'unit' were correctly classified as 'unit'. The model perfectly identified all units without confusing them with other labels.
- Near-Perfect Classification for 'quantity': Out of 980 tokens that were truly 'quantity':
 - 977 were correctly classified as 'quantity'. Only 3 were misclassified as 'unit'. This is the only type of error observed in the training data. No False Positives (mostly): The model generally avoids false positives. For example, it never incorrectly predicted 'ingredient' for a non-ingredient token. The only instance of a false positive is when it predicted 'unit' for 3 actual 'quantity' tokens.
- Overall Strong Performance: The matrix shows that the model has learned the patterns in the training data exceptionally well, with very few misclassifications.

The only challenge, albeit minor, is distinguishing between a small number of 'quantity' and 'unit' tokens.

This confirms the very high scores seen in the classification report, providing the granular detail of where the few errors occurred, which is specifically in the quantity-unit distinction.

Prediction and Model Evaluation

- **Predict and Evaluate the CRF model on validation set**

Validation Classification Report:

	precision	recall	f1-score	support
ingredient	1.00	1.00	1.00	2107
quantity	0.99	1.00	0.99	411
unit	0.99	0.99	0.99	358
accuracy			1.00	2876
macro avg	1.00	1.00	1.00	2876
weighted avg	1.00	1.00	1.00	2876

- This validation classification report demonstrates the model's performance on unseen data. Here are the key insights:
- 'ingredient' Class: The model achieves perfect precision, recall, and F1-score (1.00) for 'ingredient' tokens, which constitute a significant portion of the validation data (2107 tokens). This indicates excellent performance in identifying ingredients without false positives or negatives.
- 'quantity' Class: For 'quantity' tokens (411 support), the model shows very strong performance with 0.99 precision, 1.00 recall, and a 0.99 F1-score. The perfect recall suggests it successfully identified all actual quantity tokens, while the slightly lower precision (0.99) implies a very small number of non-quantity tokens were mistakenly classified as quantities.
- 'unit' Class: The 'unit' class (358 support) also performs very well, with 0.99 precision, 0.99 recall, and a 0.99 F1-score. This indicates that the model is highly effective at identifying units, with only a minimal number of errors in both false positives and false negatives.
- Overall Accuracy: The overall accuracy is 1.00 (or very close to it), across 2876 tokens. This is a remarkably high accuracy, suggesting the model generalizes exceptionally well to new, unseen recipe data.

- **Macro and Weighted Averages:** Both macro and weighted averages are 1.00, reinforcing the excellent performance across all classes, even considering class imbalances.

Overall Insight: The model demonstrates outstanding generalization capabilities on the validation set. While the training report showed perfect scores across the board, the validation report confirms that this performance is not due to overfitting. The very minor drops in precision and F1-score for 'quantity' and 'unit' from 1.00 to 0.99 are negligible and indicate highly robust entity recognition.

Validation Confusion Matrix:

	ingredient	quantity	unit
ingredient	2107	0	0
quantity	0	409	2
unit	0	3	355

- This validation confusion matrix provides a detailed look at how well your model performed on the unseen validation data. Here's an explanation of the results:
- **'ingredient' Class Performance:**
- **2107 True Positives:** All 2107 tokens that were truly 'ingredient' were correctly classified as 'ingredient'.
- **No Misclassifications:** The model did not misclassify any 'ingredient' tokens as 'quantity' or 'unit' (zeros in the 'ingredient' row under 'quantity' and 'unit' columns). **Insight:** This indicates perfect performance for the 'ingredient' class on the validation set, showing the model is very robust at identifying ingredients.
- **'quantity' Class Performance:**
- **409 True Positives:** Out of 411 tokens that were truly 'quantity', 409 were correctly classified as 'quantity'.
- **2 False Negatives:** 2 tokens that were truly 'quantity' were incorrectly classified as 'unit' (the '2' in the 'quantity' row, 'unit' column).
- **No False Positives:** The model did not incorrectly classify any 'ingredient' or 'unit' tokens as 'quantity'.
- **Insight:** The model performs very well for 'quantity', with only a minor confusion with 'unit' tokens.
- **'unit' Class Performance:**

- 355 True Positives: Out of 358 tokens that were truly 'unit', 355 were correctly classified as 'unit'.
- 3 False Negatives: 3 tokens that were truly 'unit' were incorrectly classified as 'quantity' (the '3' in the 'unit' row, 'quantity' column).
- No False Positives: The model did not incorrectly classify any 'ingredient' or 'quantity' tokens as 'unit' (excluding the 2 'quantity' tokens it mislabeled as 'unit').
- Insight: Similar to 'quantity', the model shows strong performance for 'unit', with a slight tendency to confuse it with 'quantity' tokens.
- Overall Interpretation:
- High Accuracy: The matrix corroborates the high accuracy reported in the classification report. The vast majority of tokens are correctly classified.
- Specific Confusion: The primary area of minor confusion is between 'quantity' and 'unit' labels. The model occasionally predicts 'unit' when the true label is 'quantity' (2 instances) and vice-versa (3 instances). This suggests some inherent ambiguity in a few tokens that can be interpreted as either a quantity or a unit, depending on subtle context or how they are explicitly defined.
- Strong Generalization: Despite these few misclassifications, the model demonstrates excellent generalization capabilities on unseen data, which is a very positive outcome, especially considering the weighting applied to these classes.

Error Analysis on Validation Data

- **Investigate misclassified samples in validation dataset**
 - Iterates through each sentence in the **validation dataset**
 - Retrieves tokens, true labels, and predicted labels for each sentence
 - Compares the true label and predicted label for **every token**
 - Identifies a **misclassification** when the labels do not match
 - Collects detailed information for each error, including:
 - The misclassified token
 - True label and predicted label
 - Previous and next tokens for local context
 - Sentence index and token index
 - Full sentence as broader context
 - Weight of the true label from weight_dict
 - Handles boundary cases where previous or next tokens may not exist
 - Stores all error details as dictionaries in an error_data list
 - Creates a comprehensive dataset for **in-depth error analysis** of the model's predictions

- **Create dataframe from error_data and print overall accuracy**
 - Overall Accuracy on Validation Data: 99.83%
 - Total misclassified tokens: 5

This result provides a clear picture of the model's overall performance on the validation dataset. Here's an analysis:

- **High Overall Accuracy:** The model achieved an impressive 99.83% accuracy on the validation data. This means that out of all the tokens in the unseen validation set, the model correctly classified nearly all of them.
- **Minimal Misclassifications:** Only 5 tokens were misclassified out of the total_tokens_val (which is 2876, as seen in previous outputs). This is an extremely low number of errors, especially for a Named Entity Recognition task.
- **Strong Generalization:** Such a high accuracy on a separate validation set is a very positive indicator. It suggests that the model has not just memorized the training data (avoiding overfitting) but has learned robust patterns that allow it to generalize exceptionally well to new, unseen recipe ingredient lists.
- **In summary:** The model is highly effective and reliable in identifying the key entities (ingredients, quantities, units) in recipe data, demonstrating excellent performance on data it has never encountered during training.

- **Provide insights from the validation dataset**

Summary of Validation Error Analysis

- **Performance:** The CRF model achieved 99.83% accuracy on the validation set, with only 5 misclassifications out of 2,876 tokens, indicating strong generalization to unseen recipe data.
- **Error Pattern:** All errors were confined to confusion between quantity and unit labels—2 quantities predicted as units and 3 units predicted as quantities. No errors involved ingredient identification.
- **Sources of Ambiguity:**
 - **Range expressions:** The token “to” in phrases like “10 to 12” was misclassified due to its dual role as a conjunction within quantity ranges.
 - **Implicit quantifiers:** Words like “a” (e.g., “a pinch”) caused confusion because they function as quantities without being explicit numbers.
 - **Context-dependent terms:** Tokens such as “whole” and “cloves” showed role ambiguity—“whole” was mistaken for a quantity, while “cloves” (a unit) was sometimes labeled as a quantity.

- **Class Weights Impact:** Despite higher penalties for quantity (7.26) and unit (8.77) compared to ingredient (0.67), misclassifications persisted within these two classes, highlighting that the errors stem from linguistic and contextual ambiguity rather than class imbalance.

Overall takeaway: The model performs exceptionally well, with remaining errors limited to inherently ambiguous quantity–unit distinctions in natural language recipe text.

Conclusion:

This assignment successfully demonstrates the development and evaluation of a Named Entity Recognition (NER) model using Conditional Random Fields (CRF) to extract key entities (ingredients, quantities, and units) from recipe data. The process involved several critical steps:

Data Ingestion and Preparation: The recipe data, provided in JSON format, was loaded into a Pandas DataFrame. Initial data cleaning involved identifying and removing rows with inconsistent token lengths between the input text and its corresponding POS tags, ensuring data integrity for subsequent steps. The data was then split into training (70%) and validation (30%) sets.

Exploratory Data Analysis (EDA): EDA was performed on both the training and validation datasets. This involved flattening tokens, categorizing them into ingredients, units, and quantities, and identifying the top 10 most frequent items for ingredients and units. The analysis revealed a consistent distribution of common ingredients (e.g., 'powder', 'Salt', 'Oil') and units (e.g., 'teaspoon', 'cup', 'tablespoon') across both datasets, suggesting a representative data split.

Feature Extraction for CRF Model: A comprehensive `word2features` function was developed to extract token-level and contextual features. These features included lexical properties (token, lemma, POS tags, shape), indicators for digits and punctuation, and improved detection for quantities and units using predefined keywords and regular expressions. Contextual features like preceding and following tokens were also incorporated. Importantly, class weights were computed using an inverse frequency method and applied during feature extraction, with the 'ingredient' label being penalized, to address potential class imbalance and prioritize accurate identification of 'quantity' and 'unit' labels.